

The image features the text "DL for ENG" in a bold, 3D, gold-colored font. The letters are floating on a deep blue ocean with visible ripples and small waves. In the background, there are rolling blue hills or mountains under a sky filled with soft, grey clouds. The overall scene has a serene, digital aesthetic.

DL for ENG

What You Will Be Able To Do

THE GOALS, IN PLAIN WORDS

- **explain** — what depth buys that width does not, and state the cost of each
- **count** — the parameters in a network before you train it
- **recognise** — underfitting and overfitting from a training curve alone
- **name** — three ways to regularise, and say when each is appropriate
- **choose** — a capacity for a problem you have not seen, and defend the choice
- **state** — why the test set is touched once

WHAT TODAY ASSUMES

from L4.1	one hidden layer, and what it computes
from L4.1	universal approximation, and its silences
not needed	how the parameters are found — that is L6.1

THE TURN IS ON SLIDE 11

Nine slides on what a bigger model can do, and then the rest of the lecture on why you should hesitate to give it one.

THE HABIT TO LEAVE WITH

Plot training and validation loss together. Every run, without exception.

Two Questions L4.1 Left Open

AND THE SECOND ONE IS THE IMPORTANT ONE

- **universal approximation left five things unsaid** — two of them are today
- **it says nothing about efficiency** — one layer suffices, but may need exponentially more units
- **and nothing about unseen data** — which affects every model you will build
- **first half** — what a bigger model can do
- **second half** — why you should be reluctant to give it one



TODAY, IN TWO HALVES

1 – 9	what depth buys: regions, reuse, cost
10	the turn — capacity as a dial
11 – 14	underfit, fit, overfit
15 – 17	bias, variance, double descent
18 – 20	regularisation, and choosing capacity
21	and why none of it applies in L7

Slides 11 to 20 used to be in L6. They are here because you need them before Ex_04 notebook 05, and because a student who cannot recognise overfitting cannot trust anything they build afterwards.

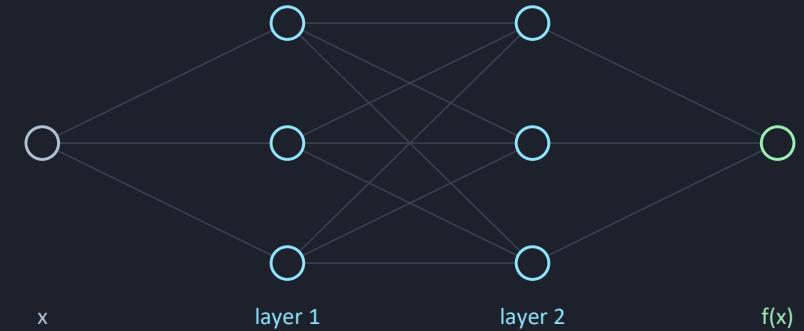
A Deep Network Is a Composition

FEED ONE NETWORK INTO THE NEXT

- **feed one shallow network into another** — that is all depth is
- **each layer computes a representation** — the next works in that representation
- **nothing new required** — no new unit, no new activation, no new loss
- the same object, applied to its own output

$$f(x) = f_K \circ f_{K-1} \circ \cdots \circ f_1(x)$$

TWO SHALLOW NETWORKS, JOINED



THE INTUITION FROM L4.1, GENERALISED

A layer transforms the space; the next layer solves an easier problem in that new space. Depth is repeated simplification, and every argument for it on the next six slides is a version of that sentence.

The Regions Multiply Rather Than Add

THE ARGUMENT, WITH SMALL NUMBERS

- a shallow ReLU net with D units — $D+1$ linear regions, one kink each
- compose — and the second layer folds the first layer's output back on itself
- K layers of D units — regions grow roughly as D to the power K
- three layers of ten — can carve more regions than a shallow net of a thousand
- that is the formal case for depth — about efficiency, not about what is possible

REGIONS AGAINST PARAMETERS



Schematic — the axes carry no units and the claim is about the shape. What is real is the ordering: at equal parameter count, depth represents more.

Matrix Notation — Bookkeeping, Not Mathematics

THE SAME THING, WRITTEN COMPACTLY

- **two subscripts per weight** — unreadable by the third layer
- **so collect them** — weights into a matrix, biases into a vector
- **nothing has changed** — the same arithmetic, written to fit on a slide
- **the activation applies elementwise** — not as a matrix operation
- **and a layer's output is the next layer's input** — nothing more

$$\mathbf{h}_k = a(\mathbf{W}_k \mathbf{h}_{k-1} + \mathbf{b}_k)$$

ONE LAYER, AS CODE

`W (D × D)` the weight matrix

`b (D)` the bias vector

`h = a(Wh + b)` one layer, entire

`nn.Linear(D, D)` PyTorch: W and b together

`nn.Tanh()` the activation, separately

Counting Parameters, and Why You Should

A HABIT WORTH FORMING NOW

- a **weight matrix between two layers of width D** — holds D squared numbers
- so width costs quadratically, depth only linearly
- **doubling width quadruples the parameters** — doubling depth merely doubles them
- print the parameter count at the start of every run
- one line, and it catches the commonest configuration mistake
- `pinn_core.py` already does this, in every Part 2 notebook

$$P = D_i D + (K - 1) D^2 + D D_o + (K D + D_o)$$

SOME REAL COUNTS

$1 \times 10 \times 1$	31 — L4.1's shallow net
$2 \times 32 \times 32 \times 1$	1,217 — a typical PINN
4 layers of 50	about 8,000 — L9's flow net
ResNet-50	25 million
a 2020 language model	175 billion

The first three are what you will build. The last two are on the slide so you can see the distance — and so that nobody concludes they need a data centre to solve a heat equation.

Deeper or Wider?

THE EXPERIMENT YOU WILL RUN THIS WEEK

Fix the parameter budget. Spend it on width, then on depth, then somewhere between. Ex_04 notebook 04 does exactly this, and the answer is not the same for every problem.

WHAT THEORY SAYS

Depth wins, sometimes enormously. There are functions needing exponentially many units in one layer that a deep network represents with a handful per layer.

The construction behind that result is a folding argument — each layer folds the input space, and folds compose multiplicatively.

But the functions used to prove it are adversarial. They were built to make the point, and nobody has shown your heat equation is one of them.

WHAT PRACTICE SAYS

Moderate depth, moderate width. Four to eight layers of twenty to fifty units solves essentially every problem in this course.

Very deep networks bring their own difficulties — vanishing gradients, and the need for residual connections and normalisation, which is L5.1.

And very wide shallow networks train easily but generalise less well at equal parameter count, which is the second half of today.

Start at 4×32 . Change it only when you have evidence.

Theory says depth can win. Practice says start moderate, measure, and let the problem tell you.

Why Depth Was Not Free

THE REASON 1986 DID NOT IMMEDIATELY GIVE US 2012

- backpropagation multiplies derivatives along the chain
- **many layers** — a long product of numbers mostly below one
- **so early layers receive almost no gradient** — and barely move
- **the vanishing gradient** — why 1986 needed twenty-six more years
- the honest answer to 'why not a hundred layers?'

GRADIENT SIZE BY LAYER



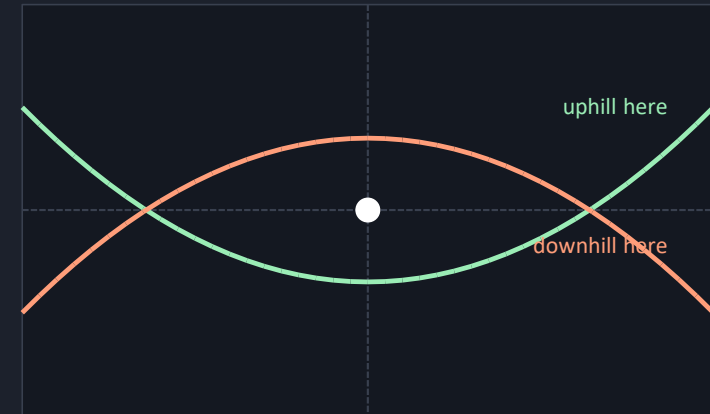
Schematic. The orange curve is why deep networks would not train before about 2010, and the green one is why they do now.

And the Landscape Gets Worse

A SURPRISE THAT WORKS IN YOUR FAVOUR

- **depth makes the loss surface far more complicated** — many minima, not one
- **you would expect that to be fatal** — it is not, and the reason is surprising
- **in high dimensions** — a vanishing gradient is almost always a saddle, not a minimum
- **a saddle has a way down** — so gradient descent tends to escape
- **and the minima it finds** — give similar loss values in practice

A SADDLE, NOT A BOWL



The gradient is zero at the white point, and it is not a minimum. In a thousand-dimensional space almost every stationary point looks like this.

Now the Question That Actually Matters

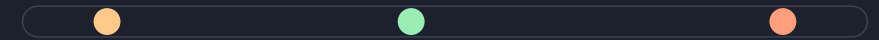
EVERYTHING SO FAR WAS ABOUT WHAT A MODEL CAN REPRESENT

- **nine slides on capability** — regions, depth, composition, cost
- **all answering one question** — how much can this model represent?
- **that is the wrong question** — and the rest of the lecture is the right one
- **you want a model that represents what generated your data** — not everything
- capacity is a dial, not a target

THE DIAL

too rigid

too flexible



a straight line

about right

a huge network



Training error falls for ever. Held-out error turns around. The green point is what you are looking for and you cannot see it without held-out data.

Too Rigid

THE MODEL CANNOT REPRESENT THE TRUTH

- **eleven noisy measurements** — the dashed curve is the truth you never see
- **fit a straight line** — two parameters, and the truth has a hump
- **underfitting** — and it is the benign failure
- **it announces itself** — the loss stops falling and stays high
- **the fix is more capacity** — and you can tell that from the training curve alone

TWO PARAMETERS



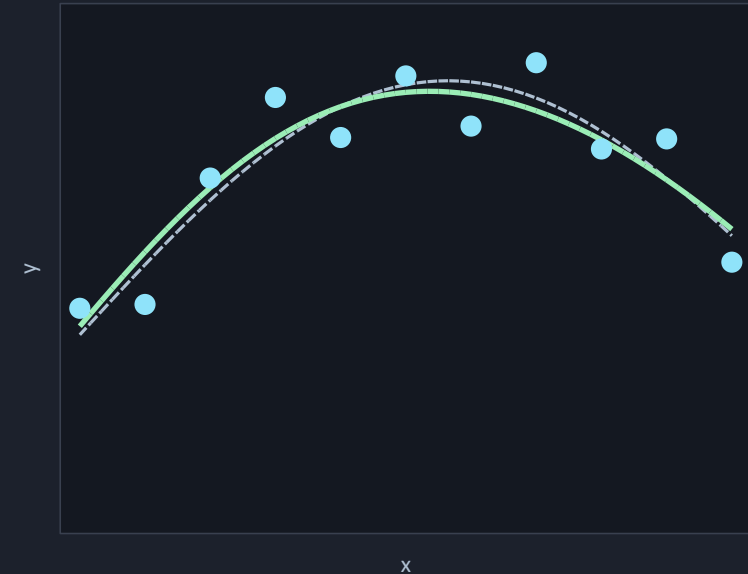
Training error high. Held-out error high. Diagnosis is easy.

About Right

ENOUGH FLEXIBILITY, AND NO MORE

- **the same eleven points** — a model of moderate capacity
- follows the trend, ignores the scatter
- **it does not pass through every point** — the scatter is not signal
- training error here is larger than on the next slide
- **this is what you are aiming for** — and it is invisible from training error alone

MODERATE CAPACITY



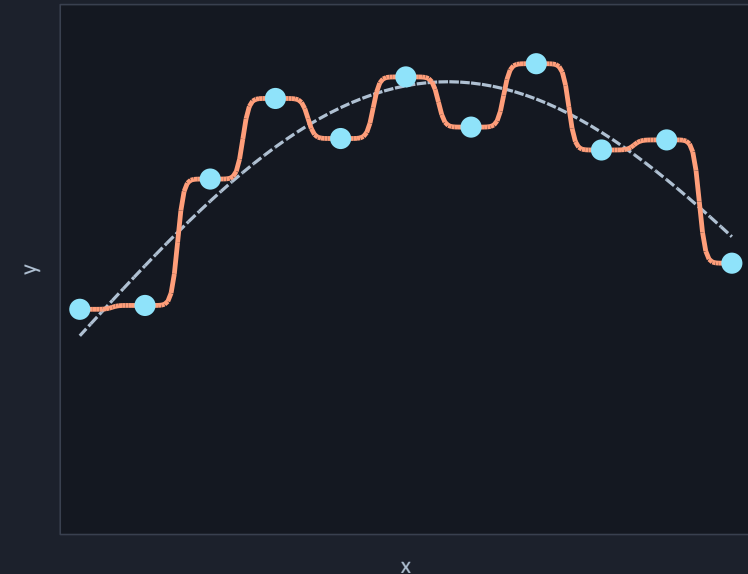
Training error moderate. Held-out error lowest. This is the target.

Too Flexible

PERFECT ON THE DATA, WRONG EVERYWHERE ELSE

- **the same eleven points** — far more capacity than the problem needs
- **it passes through every measurement** — training error essentially zero
- **look between the points** — it swings, because nothing constrains it there
- **and nothing in the output warns you** — the number looks excellent
- that is why this is the dangerous case, not merely the wrong one

FAR TOO MUCH CAPACITY



Training error ≈ 0 . Held-out error worst of the three. Loss curve: beautiful.

Hold Data Back, and Mean It

THREE SETS, THREE DIFFERENT JOBS

- **training set** — fits the parameters
- **validation set** — chooses between models: width, depth, when to stop
- **test set** — estimates performance on data nobody used to decide anything
- **touch it once** — every look and adjust turns it into a validation set
- **choosing a model on the test set** — is how published results become irreproducible



SPLITTING BADLY

random split	fine for independent samples
time series	split by time, never at random
repeated runs	split by run, not by sample
one rig, one batch	you have less data than you think
small N	cross-validate instead

The second and third rows are where engineering data most often goes wrong. Two samples from the same test run are not independent, and splitting them at random leaks the answer into the training set — you will measure a wonderful score for a model that has learned nothing.

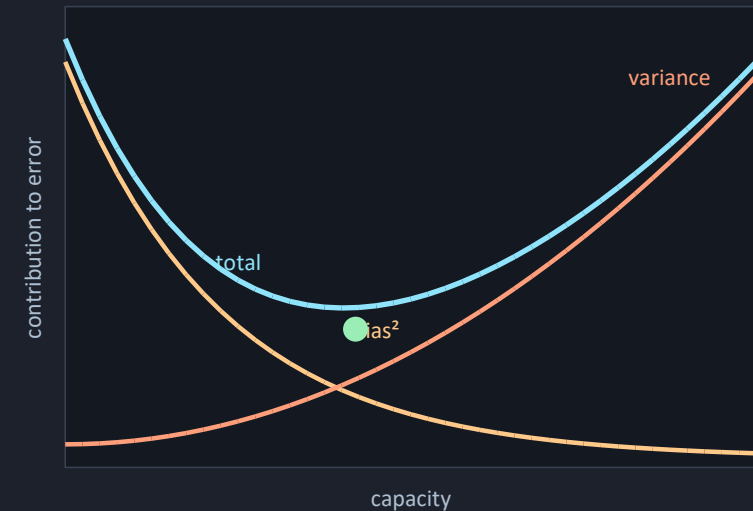
Bias and Variance

THE ERROR SPLITS INTO THREE PARTS

- **repeat the whole experiment** — new data, new noise, refit each time
- **bias** — how far the average fitted model sits from the truth
- **variance** — how much the fitted model moves when the data changes
- **noise** — the floor no model removes
- **an error below the noise floor** — means the test set leaked into training

$$\mathbb{E}[(y - \hat{f})^2] = \text{bias}^2 + \text{variance} + \sigma^2$$

THE CLASSICAL TRADE-OFF



The classical picture: a U, with a best capacity in the middle. Hold that thought for one slide.

Double Descent

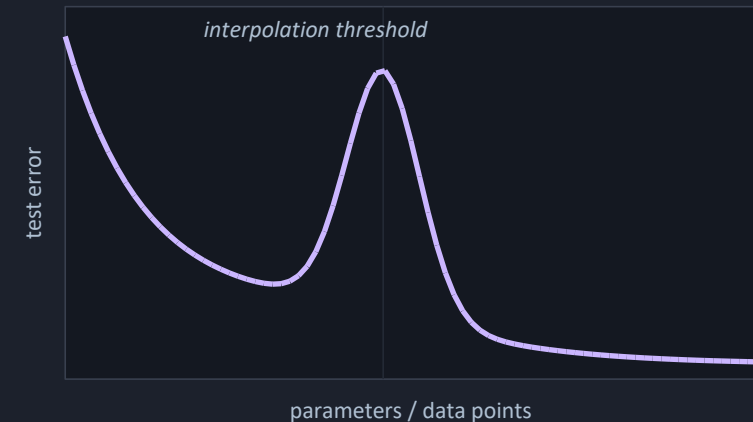
THE PICTURE ON THE LAST SLIDE IS INCOMPLETE

The classical U was derived for models with fewer parameters than data points. Modern networks routinely have far more — and when you keep going past the peak, the test error comes down again.

WHAT ACTUALLY HAPPENS

- **increase capacity from very small** — test error falls, then rises. The classical U.
- **it peaks** — around where parameters equal training points
- **keep going** — and past the peak it falls again
- **why** — among the settings that fit perfectly, descent tends to find a smooth one
- **the practical reading is narrow** — two regimes exist, and interpolating is not automatically fatal

TEST ERROR AGAINST CAPACITY



The dashed line is where the model can just fit the training data exactly.

Regularisation — Capacity You Do Not Use

FIVE THINGS, AND WHAT EACH ACTUALLY DOES

- **regularisation** — anything that discourages a flexible model from using all its flexibility
- **L2, or weight decay** — add the summed squared parameters to the loss
- **large weights become costly** — so the fitted function stays smoother
- **early stopping** — watch validation and stop when it turns. Free, and effective.
- **dropout** — switch off random units, so none can rely on another
- **data augmentation** — manufacture data with transformations that should not change the answer
- **and one nobody lists** — use a smaller model

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \lambda \sum_j \theta_j^2$$

IN THIS COURSE

L2 / weight decay yes — Ex_04 notebook 05

early stopping yes — everywhere

dropout mentioned, rarely used

augmentation L11, on camera images

a smaller model the usual answer in L7 – L12

physics in the loss L7 — the strongest of all

Look at the last row. A physics residual is a regulariser, and a far stronger one than anything above it — it does not merely discourage complexity, it forbids solutions that violate a conservation law. That is the argument of the whole second half of the course, arriving early.

Early Stopping, and the Curve You Must Plot

THE MOST INFORMATIVE PLOT IN MACHINE LEARNING

- **plot training and validation loss against epoch** — on the same axes, every run
- **both fall at first** — the model is learning real structure
- **then validation turns up** — the model is learning the noise
- **early stopping** — keep the parameters from the lowest-validation epoch
- **read the shape, not the number** — flat and high is underfitting; a widening gap is overfitting

TRAINING AND VALIDATION LOSS



Everything to the right of the dashed line is the model memorising your training set. Keep the checkpoint from the green point.

How To Actually Choose

A PROCEDURE, NOT A FORMULA

- **start deliberately too small** — underfitting tells you so, and gives you a floor
- **grow until training loss is comfortably low** — then look at validation
- **then regularise rather than shrink** — weight decay and early stopping usually win
- **random search beats grid search** — a grid wastes trials on parameters that do not matter

THE ORDER TO WORK IN

- | | |
|---|--|
| 1 | overfit five samples — is it wired up? |
| 2 | start small, note the underfit floor |
| 3 | grow until training loss is low |
| 4 | plot train and validation together |
| 5 | add weight decay, early stopping |
| 6 | only then touch the architecture |

Step one is L4.1's debugging trick and it belongs at the top of every procedure in this course. A model that cannot memorise five points has a bug, and no amount of architecture search will fix a bug.

None Of This Works When There Is No Data

THE PROBLEM, STATED PLAINLY

Everything on the last nine slides rests on holding data back. In L7 to L12 you will train networks with no labelled outputs at all. There is nothing to hold back, so train/validation/test does not exist.

WHAT STOPS WORKING

No held-out set, so no validation curve and no early stopping in the usual sense.

Overfitting does not mean what it means here. There are no training samples to memorise — the residual is evaluated at collocation points you resample at will.

The failure mode changes shape entirely: the loss goes to zero and the answer is still wrong, because the network found a function satisfying the equation but not your boundary conditions.

WHAT REPLACES IT

Residual checking: does the solution satisfy the equation at points you did not train on? That is the direct analogue of a held-out set, and it is free.

Conservation laws: does the flux balance? Does the total energy hold? L8.1 makes this the primary check, and it is worth more than any error norm.

Comparison against a classical solver on a case where one exists, which is why every Part 2 exercise has a reference implementation.

Held-out physical conditions: train at one Reynolds number, test at another.

Generalisation still exists in Part 2. It is generalisation across conditions rather than across samples.

Failure Modes of a Deep Network

THE LOSS STALLS HIGH AND FLAT

Underfitting, or a vanishing gradient, and they look identical from the loss curve. Distinguish them by shrinking the network: if a smaller model reaches the same loss, it was the gradient, not the capacity.

VALIDATION TURNS UP EARLY

Overfitting, and much sooner than students expect on small engineering datasets. Weight decay and early stopping before any architecture change.

VALIDATION IS BETTER THAN TRAINING

Almost always leakage. The same physical run appears in both splits, or you normalised using statistics computed over the whole dataset before splitting. This one is embarrassing and common.

WHAT TO CHECK, IN ORDER OF COST

free	print the parameter count
free	plot train and validation together
cheap	overfit five samples
cheap	halve the width, refit
real work	check how you split the data
real work	collect more data

Note where 'collect more data' sits. It is the right answer more often than anything above it, and it is last because it is the one you usually cannot do.

Ex_04 — Notebooks 04 and 05

TWO EXPERIMENTS, ONE ARGUMENT

- **notebook 04** — the depth-against-width sweep. Fix the budget, spend it three ways.
- **notebook 05** — the overfitting lab, on the same eleven points
- **fit a model far too large** — then regularise and measure what you recover
- **plot training and validation together** — every time
- **the report asks you to choose and defend** — the marks are for the reasoning

THE NOTEBOOKS

00	environment check — read only
01	perceptron and XOR, in NumPy
02	the same network in PyTorch
03	counting kinks — sweep D
04	depth against width, equal budget
05	overfit, then regularise

THE QUESTION THAT CARRIES THE MARKS

Which model would you deploy, and what would have to be true for that to be right?

Key Takeaways

1

Depth buys efficiency, not capability

One layer can represent anything; several can represent it with far fewer parameters. Regions multiply rather than add.

2

Capacity is a dial, not a target

Too little and you cannot represent the truth; too much and you represent the noise. There is no formula for the middle.

3

Training error is not evidence

The overfitted model had the best training loss in the lecture and was useless. Hold data back and plot both curves.

4

The classical U is not the whole story

Double descent means past the interpolation threshold test error can fall again — but you are nowhere near that regime on 400 samples.

5

Physics is the strongest regulariser

Weight decay discourages complexity. A conservation law forbids wrong answers outright. That is L7.

Next — L5.1: the architectures that exploit structure, on grids and on graphs.

Questions

TEN TO TEST WHETHER TODAY LANDED

- **depth or width** — which costs more parameters, and by how much?
- **why do the regions multiply** — rather than add, when you compose layers?
- **why did the ideas of 1986** — need twenty-six more years?
- **many local minima ought to be fatal** — why is it not?
- **a model passes through every point** — why is that not good news?
- **why is the test set touched once** — and what happens if it is not?

AND FOUR MORE

seven	name the three terms in the error decomposition
eight	what does an error below the noise floor tell you?
nine	regularise or shrink — and why?
ten	why does random search beat grid search?

THE ONE WITH NO SETTLED ANSWER

In L7 there is no validation set. So how will you know when to stop?

BEFORE L5

Run notebooks 04 and 05. Bring your overfitting plot.

References and Further Reading

THE TWO BOOKS

Prince, Understanding Deep Learning — chapter 4 for deep networks and the region-counting argument, chapter 8 for measuring performance including double descent in 8.4, chapter 9 for regularisation. Chapter 20.2 and 20.4 are the honest discussion of why any of this works.

Goodfellow, Bengio & Courville — chapter 7 is a thorough treatment of regularisation, and chapter 11, Practical Methodology, is a decision procedure for what to do when your model underperforms. There is nothing like chapter 11 in Prince and it is the better read for a working engineer.

IF YOU WANT THE SOURCE OF THE SURPRISE

Belkin et al. (2019), PNAS 116 — the paper that named double descent and made the field take it seriously.

Zhang et al. (2017), Understanding deep learning requires rethinking generalization — a network memorises random labels perfectly. Short, and it dismantles the classical account in four pages.

READ IN THIS ORDER

before Ex_04 nb 05	UDL ch. 8.1 – 8.2
if you supervise projects	GBC ch. 11 — all of it
if slide 16 bothered you	Belkin et al. (2019)
before L5.1	UDL ch. 10.1 – 10.2
for the exercise	the run guide in Ex_04

Still Owed: How Any Of This Is Fitted

TWO LECTURES ON MODELS, NONE ON TRAINING

- **you now know what a deep network is** — and how much capacity to give it
- **what is still missing** — how the parameters are actually found
- **that is L6.1** — where the loss comes from, and how gradients are computed
- **it also contains four slides** — that make the whole second half possible
- **and between here and there** — L5, the architectures

WHERE YOU ARE

L3 what AI is, and what it built

L4 the model — done today

L5 architectures with structure

L6 training, and after

L7 – L12 physics in the loss

Two of five blocks complete. Everything so far has been general deep learning — recognisable in any course anywhere. From L5 the material starts bending towards what this course is actually for.

NEXT

L5.1 — Convolutional and Graph Networks

So far every layer has connected everything to everything. Next time we use the structure of the input instead — one learned local rule applied everywhere, on a grid and then on a graph. It is the same idea twice, and it is what GraphCast runs on.

BEFORE THE NEXT SESSION

read	UDL ch. 10.1 – 10.2
run	Ex_04 notebooks 04 and 05
bring	your train/validation curve from nb 05
think about	why a fully connected layer on an image is wasteful

Bring the curve. The gap between your two lines is the single most useful thing you will produce this session, and L5.1 opens by asking three people to describe the shape of theirs.